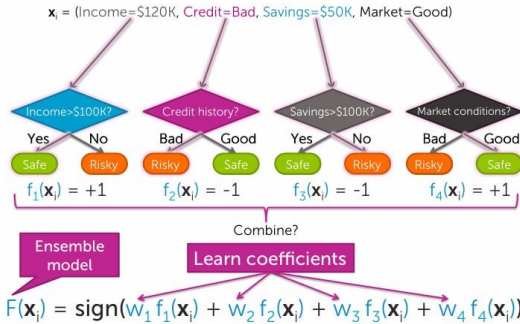


Boosting

- Simple (weak) classifiers are good. They have low variance, learning is fast. But they have high bias.

Boosting is a general ensemble method in machine learning that aims to create a strong classifier or regressor by combining the predictions of several weaker models. The idea is to build the strong model incrementally, by sequentially adding weak models that are trained to correct the mistakes made by the existing ensemble.

Ensemble methods: Each classifier "votes" on prediction



Ex: $w_1 = 2, w_2 = 1.5,$
 $w_3 = 1.5, w_4 = 0.5$

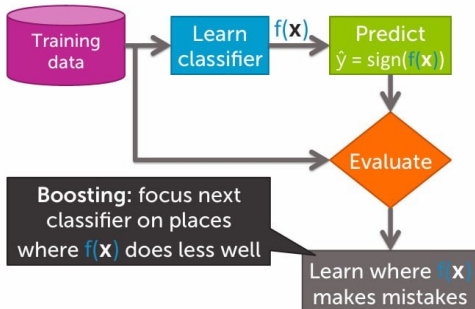
$$F(x_i) = \text{sign}(2x_1 + 1.5 \times (-1) + 1.5 \times (-1) + 0.5 \times 1)$$

$$= \text{sign}(-0.5) = -1$$

$$F_T(x) = \sum_{t=1}^T f_t(x) \quad \text{where } f_t(x) = w_t h_t(x)$$

coefficient classifier

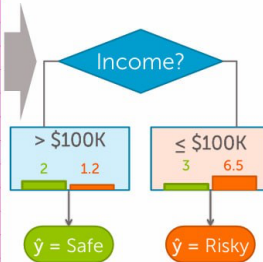
Boosting = Focus learning on "hard" points



Learning a decision stump on weighted data

Increase weight α of harder/misclassified points

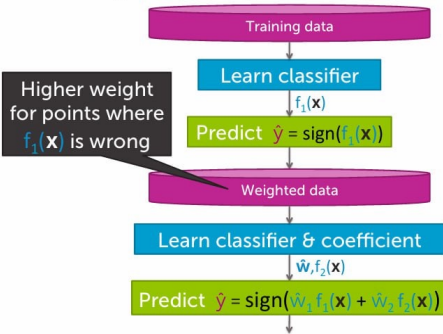
Credit	Income	y	Weight α
A	\$130K	Safe	0.5
B	\$80K	Risky	1.5
C	\$110K	Risky	1.2
A	\$110K	Safe	0.8
A	\$90K	Safe	0.6
B	\$120K	Safe	0.7
C	\$30K	Risky	3
C	\$60K	Risky	2
B	\$95K	Safe	0.8
A	\$60K	Safe	0.7
A	\$98K	Safe	0.9



Ex. Gradient ascent for logistic regression

$$w_j^{(t+1)} \leftarrow w_j^{(t)} + \eta \sum_{i=1}^N \alpha_i h_j(x_i) \left(I(y_i = +1) - P(y = 1 | x_i, w^{(t)}) \right)$$

Boosting = Greedy learning ensembles from data



How would you classify an email as SPAM or not? Like everyone else, our initial approach would be to identify 'spam' and 'not spam' emails using following criteria. If:

1. Email has only one image file (promotional image), It's a SPAM
2. Email has only link(s), It's a SPAM
3. Email body consist of sentence like "You won a prize money of \$ xxxxxx", It's a SPAM
4. Email from our official domain "[Analyticsvidhya.com](https://www.analyticsvidhya.com)", Not a SPAM
5. Email from known source, Not a SPAM

Above, we've defined multiple rules to classify an email into 'spam' or 'not spam'. But, do you think these rules individually are strong enough to classify an email successfully? No.

Individually, these rules are not powerful enough to classify an email into 'spam' or 'not spam'. Therefore, these rules are called as **weak learner**.

Layman:-

To convert weak learner to strong learner, we'll combine the prediction of each weak learner using methods like:

- Using average/ weighted average
- Considering prediction has higher vote

For example: Above, we have defined 5 weak learners. Out of these 5, 3 are voted as 'SPAM' and 2 are voted as 'Not a SPAM'. In this case, by default, we'll consider an email as SPAM because we have higher(3) vote for 'SPAM'.

Ada Boost

1) start same weights for all points $\alpha_i = \frac{1}{N}$.

2) For $t = 1, 2, \dots, T$

— Learn $f_t(x)$ with data points α_i

— compute coefficients $\hat{w}_t$

— Recompute weights α_i

3) Final model predicts by :

$$\hat{y} = \text{sign} \left(\sum_{t=1}^T \hat{w}_t f_t(x) \right)$$

Computing coefficient $\hat{w}_t$ of classifier $f_t(x)$

Total weight of mistakes = $\sum_{i=1}^N \alpha_i I(\hat{y}_i \neq y_i)$

Total weights of all points = $\sum_{i=1}^N \alpha_i$

weighted error = $\frac{\text{Total weight of mistakes}}{\text{Total weight of all points}}$

Best = 0 , Worst = 1

$$\hat{w}_t = \frac{1}{2} \ln \left(\frac{1 - \text{weighted error}(f_t)}{\text{weighted error}(f_t)} \right)$$

Is $f_t(x)$ good?		weighted_error(.) on training data	$\frac{1 - \text{weighted_error}(.)}{\text{weighted_error}(.)}$	$\hat{w}_t$
	Yes	0.01	$\frac{1 - 0.01}{0.01} = 99$	$\frac{1}{2} \ln 99 = 2.3$
	No	0.5	$\frac{1 - 0.5}{0.5} = 1$	0
		0.99	$\frac{1 - 0.99}{0.99} = 0.01$	-2.3

Terrible classifier, but $1 - f_t$ is awesome !!!

Formula of updating weights

$$\alpha_i = \begin{cases} \alpha_i e^{-\hat{w}_t} & \text{if } f_t(x_i) = y_i \\ \alpha_i e^{\hat{w}_t} & \text{if } f_t(x_i) \neq y_i \end{cases}$$

Did f_t get x_i right?		$1(x_i = y_i)$	$\hat{w}_t$	Multiply α_i by	Implication
	Yes	Correct	2.3	$e^{-2.3} = 0.1$	Decrease importance of x_i, y_i
		Correct	0	$e^0 = 1$	Keep importance the same
	No	Mistake	2.3	$e^{2.3} = 9.98$	Increasing importance of x_i, y_i
		Mistake	0	$e^0 = 1$	Keep importance the same

Normalizing weights

- If x_i often mistake, weights α_i gets very large

- If x_i often connect, weights α_i gets very small.

Normalize weights to add up to 1 after every iteration

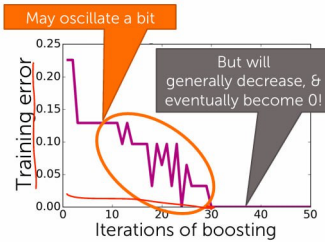
$$\alpha_i \leftarrow \frac{\alpha_i}{\sum_{j=1}^N \alpha_j}$$

AdaBoost Theorem

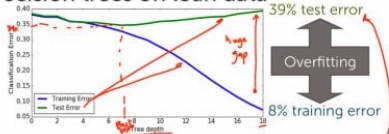
Under some technical conditions...



Training error of
boosted classifier $\rightarrow 0$
as $T \rightarrow \infty$



Decision trees on loan data



Boosted decision stumps on loan data

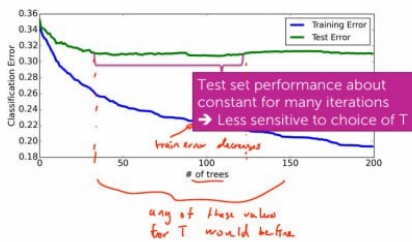


62

©2015-2016 Emily Fox & Carlos Guestrin

Machine Learning Specialization

Boosting tends to be robust to overfitting

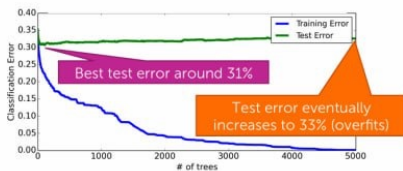


63

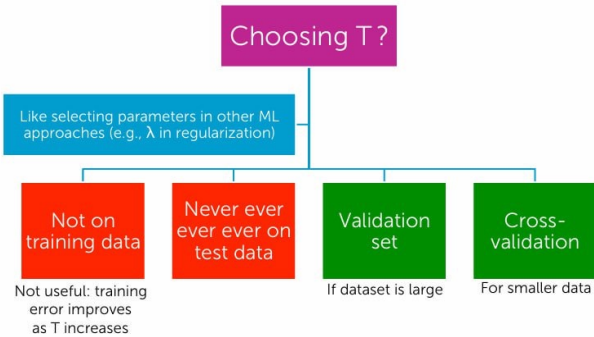
©2015-2016 Emily Fox & Carlos Guestrin

Machine Learning Specialization

But boosting will eventually overfit,
so must choose max number of components T



How do we decide when to stop boosting?



Hyperparameters

- i) n -estimators : No of sequential trees to be built.
- ii) learning-rate :
 - Scales the contribution of each tree in the final ensemble.
 - Forcing to learn more slowly.
 - values typically are 0.01, 0.3.
- iii) max-depth : controls the maximum depth of each individual decision tree.
It is usually kept low.



Gradient Boosting

Gradient Boosting is a machine learning ensemble technique that aims to build a strong predictive model by combining the predictions of several weaker models using the concept of Additive Modelling, typically decision trees. The method works by iteratively adding models to the ensemble, with each new model trained to correct the mistakes made by the combined ensemble of existing models.

Gradient Boosting is a powerful and flexible method that can be used for both regression and classification tasks.

It is particularly effective when the data has complex, non-linear relationships, and it often performs well even with little hyperparameter tuning.

Its popularity in real-world applications and machine learning competitions is testament to its effectiveness, and it has implementations in most major machine learning libraries, such as scikit-learn, XGBoost, LightGBM, and CatBoost.

Input: training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$, number of iterations M .

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.

2. For $m = 1$ to M :

(a) For $i = 1, 2, \dots, N$ compute

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}.$$

pseudo residual

(b) Fit a regression tree to the targets r_{im} giving terminal regions

$$R_{jm}, j = 1, 2, \dots, J_m.$$

↳ Leaf Nodes

(c) For $j = 1, 2, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

(d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.

①

$$L(y_i, \hat{y}_i) = \frac{1}{2} (y_i - \hat{y}_i)^2$$

$$f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$$

$$L(y_i, \gamma) = \frac{1}{2} (y_i - \gamma)^2$$

$$\frac{\partial f_0(x)}{\partial \gamma} \stackrel{\text{set}}{=} 0$$

$$\Rightarrow \frac{\partial}{\partial \gamma} \sum_{i=1}^n \frac{(y_i - \gamma)^2}{2} \stackrel{\text{set}}{=} 0$$

$$\Rightarrow \sum_{i=1}^n (y_i - \gamma) = 0$$

$$\Rightarrow \boxed{\gamma = \bar{y}}$$

$$\therefore f_0(x) = \bar{y}$$

② $m = 1 \rightarrow$ First decision tree

a) $i = 1, 2, \dots, N$

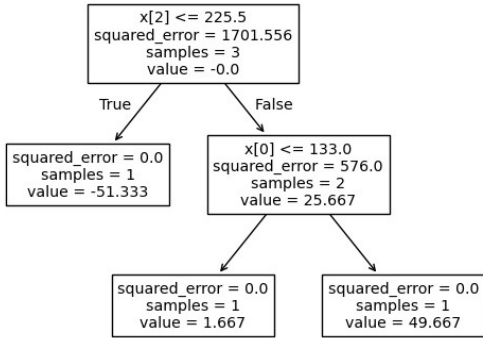
$$r_{i1} = - \frac{\partial L(y_i, f_0(x))}{\partial f_0(x)}$$

$$= - \frac{\partial (y_i - f_0(x))^2}{\partial f_0(x)}$$

$$= y_i - f_0(x) = y_i - \bar{y}$$

$$\therefore r_{i1} = y_i - \bar{y}, \quad i = 1(1)N$$

b) Fit regression tree on target y_i , $i=1(1)N$.



$$r_{11} = \arg \min_{\gamma} (y_3 - f_0(x_3) - \gamma)^2$$

$$\Rightarrow r_1 = y_3 - f_0(x_3)$$

Note:

For SEL, r_{jm} becomes the prediction by the m th decision tree.

$$d) f_m(x) = f_{m-1}(x) + \underbrace{\sum_{j=1}^{J_m} r_{jm} I(x \in R_{jm})}_{\text{output of the decision tree}}$$

output of the decision tree

$$f_1(x) = f_0(x) + dt_1$$

$$f_2(x) = f_1(x) + dt_2 = f_0(x) + dt_1 + dt_2$$

$$f_3(x) = f_2(x) + dt_3 = f_0(x) + dt_1 + dt_2 + dt_3$$

$\vdots$

$$f_M(x) = f_{M-1}(x) + dt_M = f_0(x) + dt_1 + dt_2 + dt_3 + \dots + dt_M$$

$$3) \text{ output } \hat{f}(x) = f_M(x)$$

	R&D	Ops	Marketing	Profit
0	165	137	472	192
1	101	92	250	144
2	29	127	201	91

$f_0(x)$	$y - f_0(x)$	$f_1(x)$	$y - f_1(x)$	$f_2(x)$

Classification :

$$L = - \left\{ y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right\}$$

$$1) f_0(x) = \log_e(\text{odds}) = \log_e\left(\frac{\#1}{\#0}\right) \quad (\text{pred } 1)$$

$$\text{probability } 1 = \frac{1}{1 + e^{-\log_e(\text{odds})}}$$

set a threshold (say 0.5) $\begin{matrix} > 0.5 \rightarrow 1 \\ & \searrow \\ & \leq 0.5 \rightarrow 0 \end{matrix}$

$$2) a) \text{res}_i = y_i - \text{probability } 1, \quad i = 1 \dots N$$

$\downarrow$
cont.

b) fit a decision tree on $\text{res}_i, i = 1 \dots N$
(reg. tree)

Note: the predictions from decision tree is probability. we can't directly add it to $f_0(x) = \log(\text{odds})$.
we have to convert it odds.

c) calculate $\log(\text{odds})$ of each leaf node.

Σ Residuals

$$\Sigma [\text{Previous Prob} * (1 - \text{Previous Prob})]$$

(pred 2)

for each data point

$f_0(x) + \log(\text{odds})$ of the
leaf node in which
this data point falling

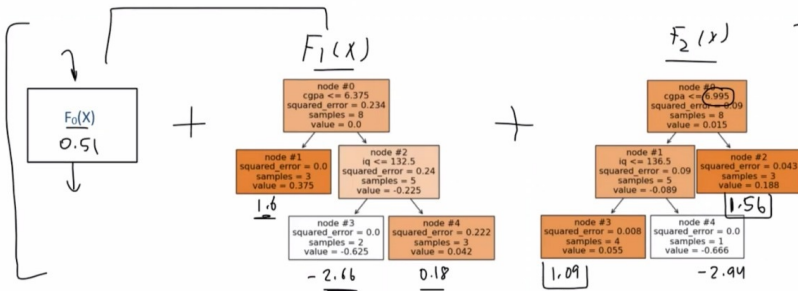
calculate probability-2

$$\text{residual 2 } (r_{i2}) = y_i - \text{Probability 2}$$

Note: To slow down the learning we can multiply $\log(\text{odds})$ to a learning rate.

Prediction:-

New data point = $\{7.2, 100\}$



$$0.51 + (-2.66) + 1.56 = -0.59$$

$$\text{prob} = \frac{1}{1 + e^{-(-0.59)}} = 0.35$$

XGBOOST

- It's a library not any algorithm.

Advantages of XGBoost :

Regularization :- Standard GBM implementation has no

regularization like XGBoost ; therefore it also helps to reduce overfitting. In fact XGBoost is also known as 'regularized boosting' technique.

Parallel Processing :-

XGBoost implements parallel processing and is faster as compared to GBM.

while trees are still built sequentially, the process of finding best split for each node is parallelized. Finding the best split for a node is computationally intensive task. It involves evaluating every possible split point for every feature. XG Boost parallelizes this very step. when building a tree, XG Boost can evaluate different features simultaneously.

Missing values :-

XGBoost has an in-built routine to handle missing values.

Handling sparse data :- Missing values on data processing steps like one-hot encoding make data sparse. XG-Boost algorithm incorporates a sparsity-aware split finding algorithm to handle different types of sparsity in the data.

- **Block Structure for Data:** XGBoost organizes the data in memory into a special structure called a "block." This block is sorted by feature values, which allows for the efficient, parallel scanning of columns to find the best split points.
- **Cache-Aware Access:** The algorithm is designed to be aware of the computer's memory hierarchy (CPU caches). By accessing data in a way that maximizes the use of these faster caches, it minimizes the time spent waiting for data from the slower main memory.
- **Out-of-Core Computation:** XGBoost can handle datasets that are too large to fit into memory by splitting the data into blocks and storing them on disk. It can then process these blocks in parallel.

- Flexibility:- It supports variety of datatypes, objective functions, regression, classification and ranking problems.
- Supports Distributed Computing.
- Supports GPU

Regression :-

I) The steps are exactly same as GBM but the decision trees made for predicting errors of previous weak learner is different

Ex:

<u>cgpa</u>	<u>package</u>	<u>model1</u>	<u>residual1</u>
6.7	4.5	7.3	-2.8
9.0	11.0	7.3	3.7
7.5	6.0	7.3	-1.3
5.0	8.0	7.3	0.7

residual 1 $\overset{DT}{\sim}$ cgpa

Similarity Score = $\frac{(\text{Sum of residuals})^2}{\# \text{ residuals} + \lambda}$

↳ regularization parameter

(at $\lambda=0$)

$$SS = \frac{(-2.8 + 3.7 - 1.3 + 0.7)^2}{4 + 0} = 0.02$$

- Similarity Score is used as splitting criteria in DT.

Cgpa

5.0 } → 5.85
6.7 } → 7.1
7.5 } → 8.1
9.0 }

Residual 1

0.7
- 2.8
- 1.3
3.7

Splitting Criteria-①

5.85

Cgpa < 5.85

0.7

- 2.8, -1.3
3.7

$$SS_L = \frac{0.7^2}{1} = 0.49$$

$$SS_R = \frac{2.8^2 + 1.3^2 + 3.7^2}{3} = 0.05$$

$$\begin{aligned} \text{gain} &= (SS_L + SS_R) - SS_{\text{root}} \\ &= 0.49 + 0.05 - 0.02 \\ &= 0.52 \end{aligned}$$

Splitting Criteria-①

7.1

Cgpa < 7.1

0.7, -2.8

-1.3, 3.7

$$SS_L = \frac{(0.7 - 2.8)^2}{2} = 2.20$$

$$SS_R = \frac{(-1.3 + 3.7)^2}{2} = 2.88$$

$$\begin{aligned} \text{gain} &= 2.20 + 2.88 - 0.02 \\ &= 5.06 \end{aligned}$$

Splitting Criteria-①

8.1

Cgpa < 8.25

0.7, -2.8, -1.3

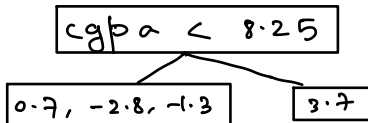
3.7

$$SS_L = \frac{(0.7 - 2.8 - 1.3)^2}{2} = 3.85$$

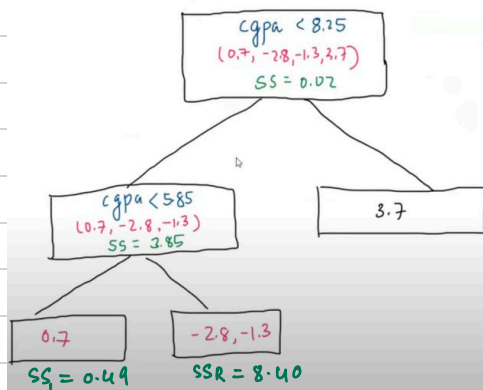
$$SS_R = \frac{(3.7)^2}{1} = 13.69$$

$$\begin{aligned} \text{gain} &= 3.85 + 13.69 - 0.02 \\ &= 17.52 \end{aligned}$$

Stage-2 :

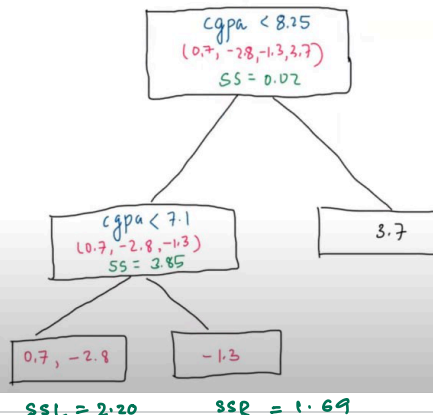


① Splitting Criteria $\rightarrow 5.85$



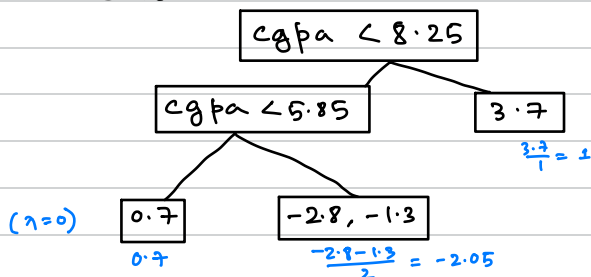
$$\text{gain} = 0.49 + 8.40 - 3.85 = 5.04$$

② Splitting criteria $\rightarrow 7.1$



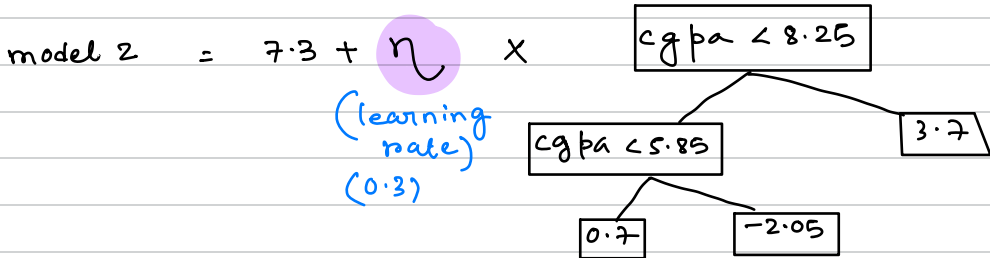
$$\text{gain} = 2.20 + 1.69 - 3.85 = 0.04$$

The final decision tree



Output = $\frac{\text{Sum of residuals}}{\# \text{ residuals} + \lambda}$
(leaf node)

<u>cgpa</u>	<u>package</u>	<u>model 1</u>	<u>residual 1</u>	<u>model 2</u>	<u>residual 2</u>
6.7	4.5	7.3	-2.8	6.69	-2.19
9.0	11.0	7.3	3.7	8.41	2.59
7.6	6.0	7.3	-1.3	6.69	-0.69
5.0	8.0	7.3	0.7	7.59	0.49



classification

$$\log\left(\frac{p}{1-p}\right)$$

p : prob. of true class

Problem statement:

cgpa	placed	$\log(\text{odds})$	$\frac{1}{1+e^{-\log(\text{odds})}}$ prob1
5.7	0	$\log\left(\frac{3/5}{2/5}\right) = 0.405$	$\frac{1}{1+e^{-0.405}} = 0.6$
6.25	1	0.405	0.6
7.10	0	0.405	0.6
8.15	1	0.405	0.6
9.60	1	0.405	0.6

$$\begin{array}{r} \text{resid 1} \\ -0.6 \end{array}$$

$$\text{resid 1} \stackrel{\text{DT}}{\sim} \text{cgpa}$$

$$0.4$$

$$-0.6$$

$$0.4$$

$$0.4$$

$$\text{similarity score} = \frac{(\sum \text{residual}_i)^2}{\sum (\text{prev prob}_i)(1 - \text{prev prob}_i) + \lambda}$$

$$\text{SS} = \frac{(-0.6 + 0.4 - 0.6 + 0.4 + 0.4)^2}{5 \times 0.6 \times (1 - 0.6) + 0} = 0$$

cgpa	res1
5.70	-0.6 ✓
6.25	0.4
7.10	-0.6
8.15	0.4
9.60	0.4

→ 5.97
 → 6.67
 → 7.62
 → 8.87

Splitting criteria = 5.97

cgpa < 5.97 ✓

-0.6
 SS_L
 $\frac{(-0.6)^2}{0.6(1-0.6)}$
 $= \frac{0.36}{0.24}$
 $= 1.5$

0.4, -0.6, 0.4, 0.4
 SS_R
 $\frac{(0.4 - 0.6 + 0.4 + 0.4)^2}{4 \times 0.6 \times 0.4}$
 $\frac{0.36}{4 \times 0.24} = \frac{1.5}{4}$
 $= 0.37$

gain = (SS_L + SS_R) - SS_{joint}
 $= (1.5 + 0.37) - 0$
 $= 1.87$

Splitting criteria = 6.67

cgpa < 6.67 ✗

-0.6, 0.4
 SS_L
 $\frac{(-0.2)^2}{2[0.6 \times 0.4]}$
 $= \frac{0.04}{2 \times 0.24}$
 $= \frac{0.04}{0.48} = 0.08$

-0.4, 0.4, 0.4
 SS_R
 $\frac{(0.2)^2}{3 \times 0.24}$
 $= \frac{0.04}{3 \times 0.24}$
 $= 0.05$

gain = 0.08 + 0.05 - 0
 $= 0.13$

Splitting criteria = 7.62

cgpa < 7.62

-0.6, 0.4, -0.6
 $\frac{(-0.8)^2}{3 \times 0.24}$
 $= 0.88$

0.4, 0.4
 $\frac{(0.8)^2}{2 \times 0.24}$
 $= 1.33$

gain = 0.88 + 1.33 - 0
 $= 2.22 \rightarrow$

Splitting criteria = 8.87

cgpa < 8.87

-0.6, 0.4, -0.4, 0.4

0.4

Stage 2 model:

cgpa < 7.62

-0.6, 0.4, -0.6

$\frac{-0.6 + 0.4 - 0.6}{3 \times 0.6 \times 0.4} = -1.11$

0.4, 0.4

$\frac{0.4 + 0.4}{2 \times 0.6 \times 0.4} = 1.66$

(Note: we can further split the leaf nodes)

Output = $\frac{\sum \text{residual}_i}{\sum p_{\text{prev-prob}_i} (1 - p_{\text{prev-prob}_i}) + \lambda}$

cgpa	placed?	pred1(lo)	pred1(prob)	yes1	pred2(lo)	pred2(prob)	yes2
5.70	0	0.405 ✓	0.6	-0.6	0.072	0.918	-0.51
6.25	1	0.405	0.6	0.4	0.072	0.918	0.48
7.10	0	0.405	0.6	-0.6	0.072	0.918	-0.51
8.15	1	0.405	0.6	0.4	0.903	0.712	0.28
9.60	1	0.405	0.6	0.4	0.903	0.712	0.28

$\text{pred2}_{(\log \text{odds})} = \text{pred1}_{(\log \text{odds})} + \eta \times \text{model2}$

















